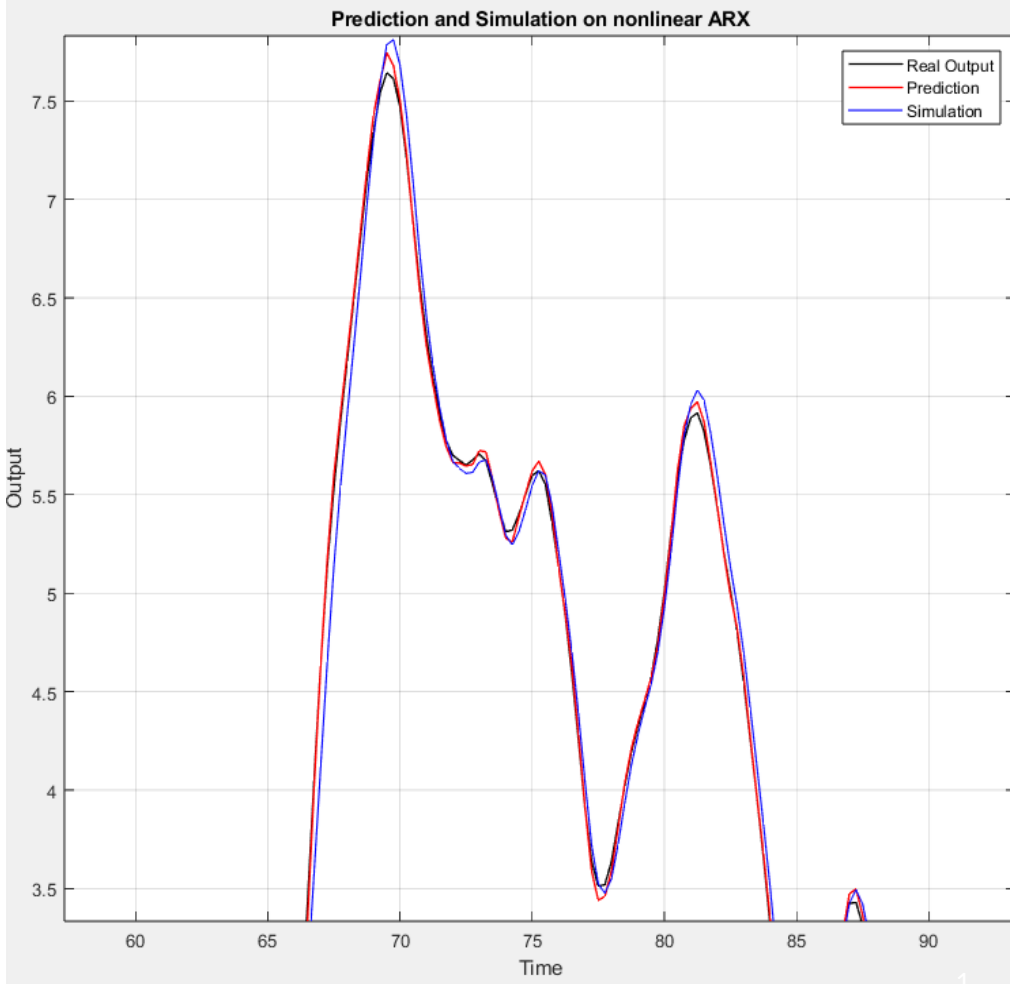
Analiza unui model
ARX Neliniar

Moroșanu Ioan Teodor
Moroșan Sebastian-Vasile
Nagy David-Csaba

P.id: 8/7



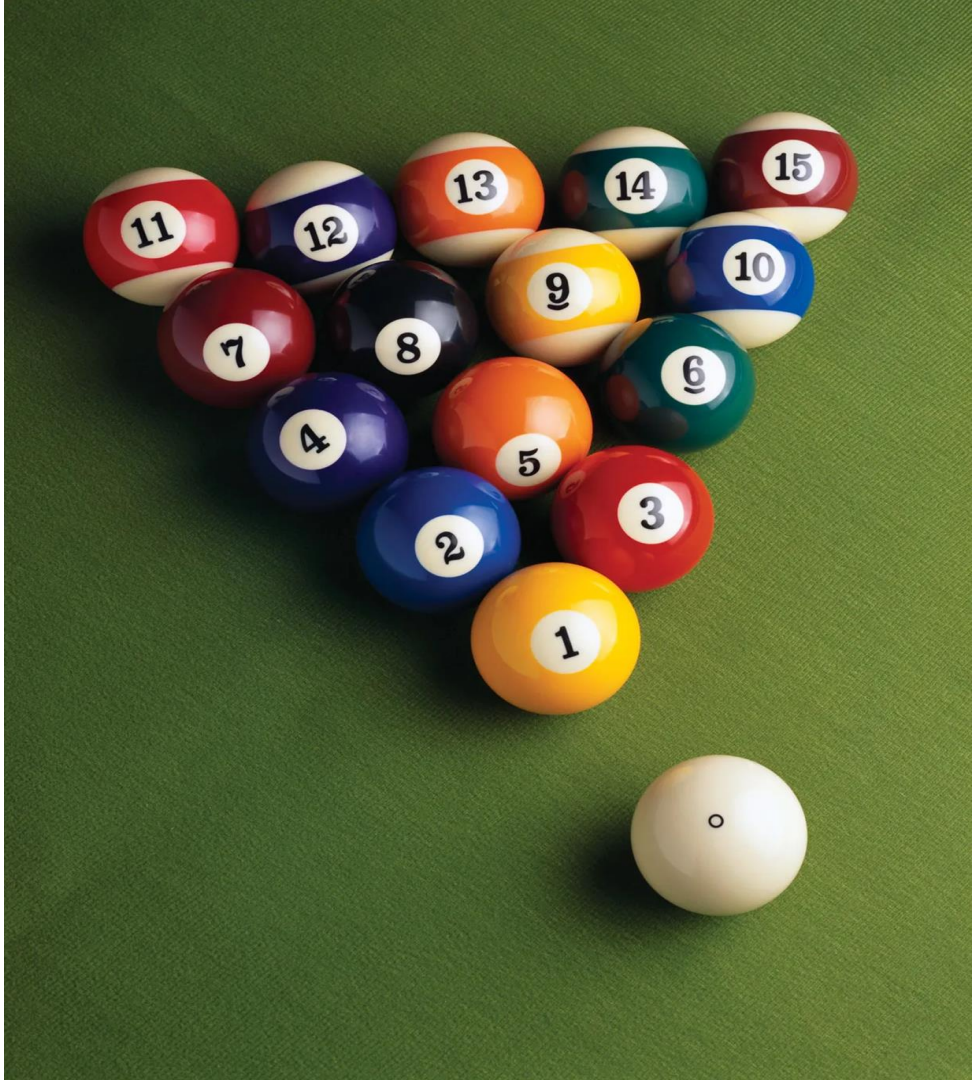
1. Problemă vs Soluție

→ **Probleme:**

- Sistem dinamic cu **comportament neliniar**, relație intrare–ieșire dificil de **descris analitic**, erori mari în simulare pentru **date noi.**

→ **Soluție:**

- Model ARX neliniar (**NARX**), utilizarea **regresorilor polinomiali**, validare pe set de date **independent.**



2. Descrierea soluției

1) <u>Structura aproximatorului:</u>

- Model **ARX neliniar (NARX) polinomial.**
- Ecuația generală: $\hat{y}(k) = p(y(k-1), \dots, y(k-n_a), u(k-1), \dots, u(k-n_b))$
- Regresori: construiți din **ieșiri și intrări întârziate** și **termeni polinomiali**, formează **vectorul de regresori** și matricea Φ

2. Descrierea soluției

2) <u>Procedura de găsire a parametrilor ϑ</u>:

- Se construiește **matricea de regresie** Φ_{id}
- Se rezolvă sistemul:

$$\Phi_{id} * \theta = Y_{id}$$

- Deși modelul este neliniar în variabile, acesta este **liniar în parametrii**, permițând identificarea eficientă

3. Particularități

- Normalizarea datelor de intrare: două metode diferite de normalizare pentru a evita erorile numerice.

- Alegerea ordinelor optime ale intrării și a ieșirii pentru a modela cât mai fidel sistemul.

- Analiza rezultatelor obținute de predicție și simulare pentru a stabili fidelitatea modelului obținut.

3. Particularități

1) Normalizarea datelor de intrare:
   - a) Trecerea valorilor din domeniul original în intervalul **[-1, 1]** pentru a rezolva mai rapid **operațiile de ridicare la putere**.

   - b) Împărțirea valorilor la **numărul total de eșantioane** pentru obține o stabilitate mai bună a modelului.

3. Particularități

2) <u>Alegerea ordinelor optime</u>:
   - Atât pentru **intrări cât și pentru ieșiri** s-au ales ordinele (n_a - n_b) cele mai optime pentru a modela cât mai fidel sistemul.

   - **Ordinul sistemului** a fost ales maxim **3** pentru a evita cât mai mult instabilitățile numerice.

3. Particularități

3) Analiza rezultatelor obținute (predicție - simulare):
   - **Predicția** prezintă niște rezultate optimiste, fiind strâns legată cu ieșirea de validare a sistemului (nu este o reprezentare fidelă a realității).

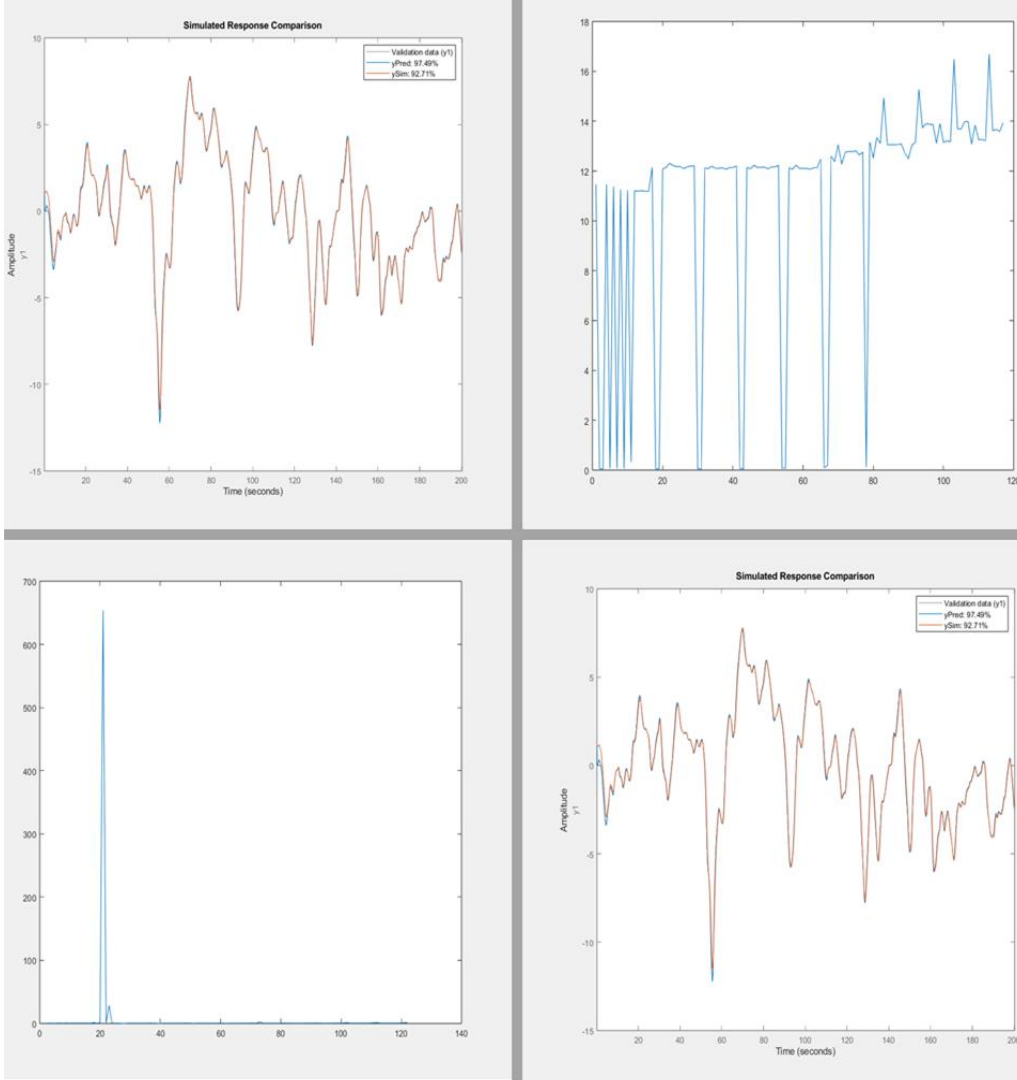
   - **Simularea** prezintă rezultate înclinate spre realitate, bazându-se pe simulările anterioare, lucru ce o face mai ideală când vine vorba de fidelitatea modelului.

4. Rezultate

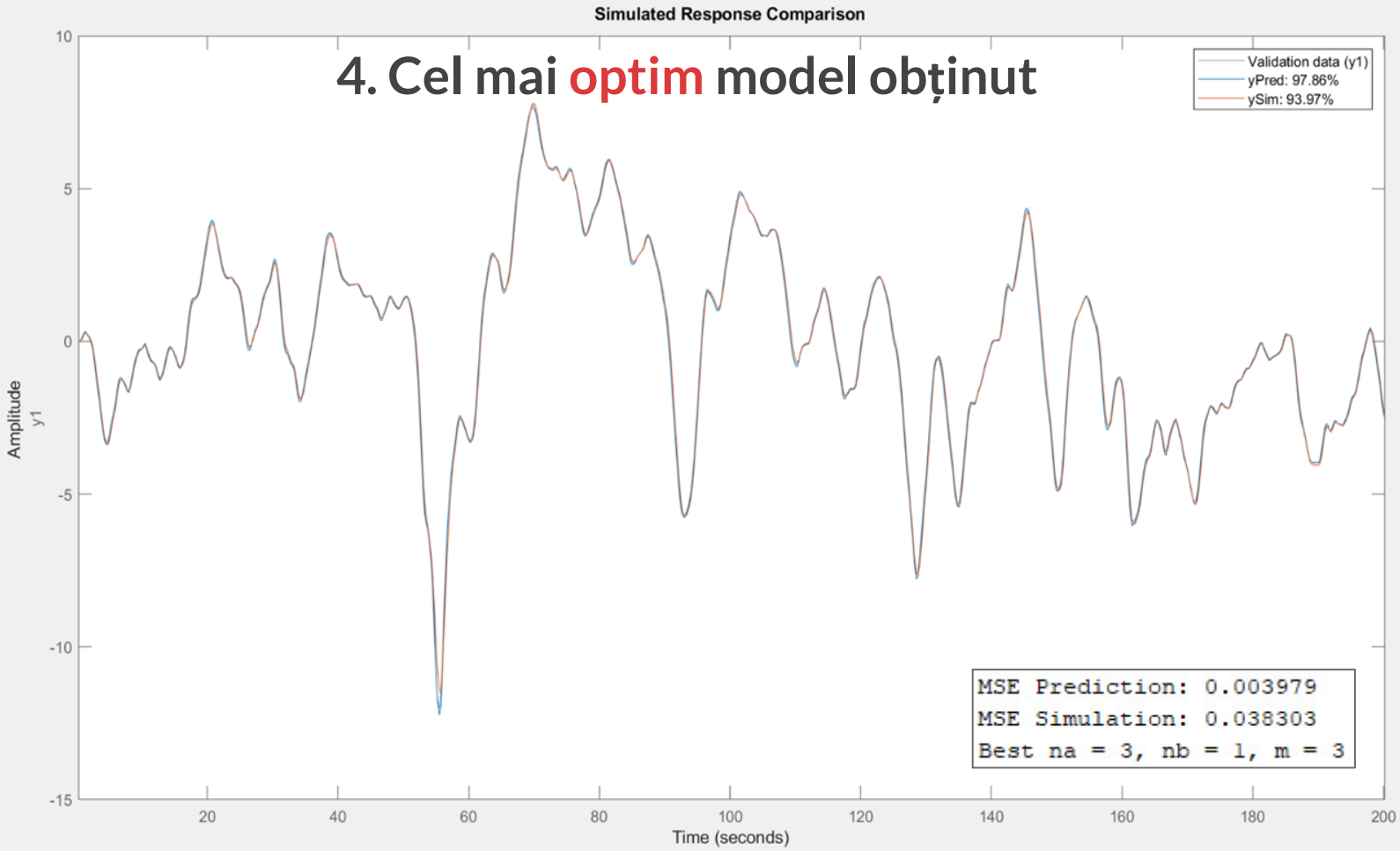
- Modelul obținut în urma celei mai **optime simulări** și **gradele de interes.**

- **Prezentarea rezultatelor** în funcție de metoda de normalizare a datelor de intrare.

- Observații relevante (**instabilități**).

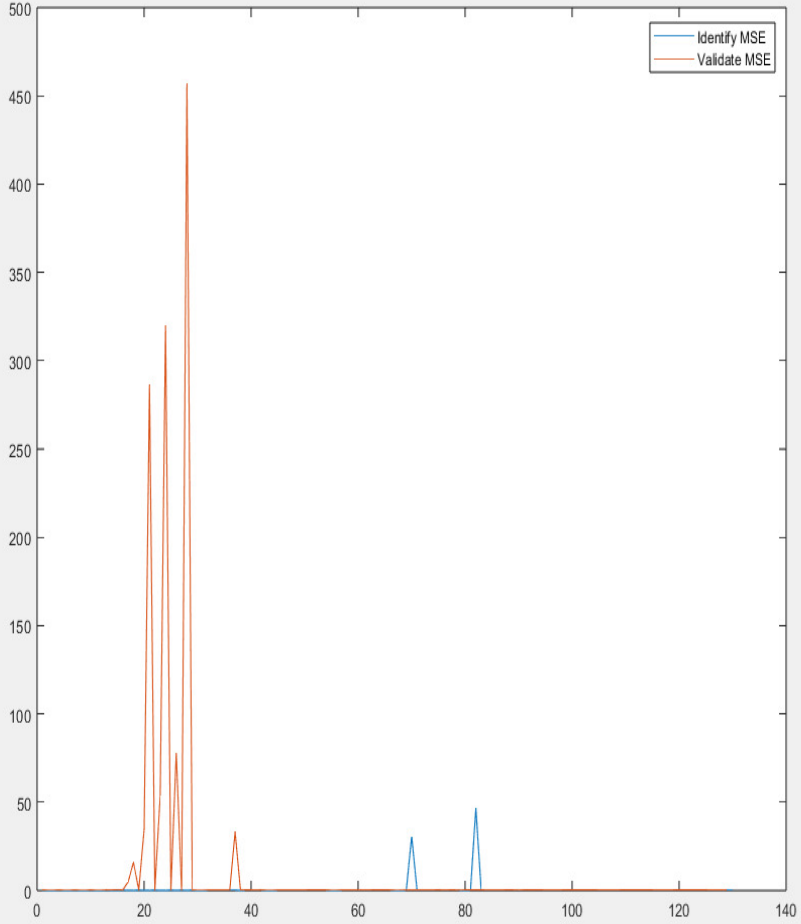


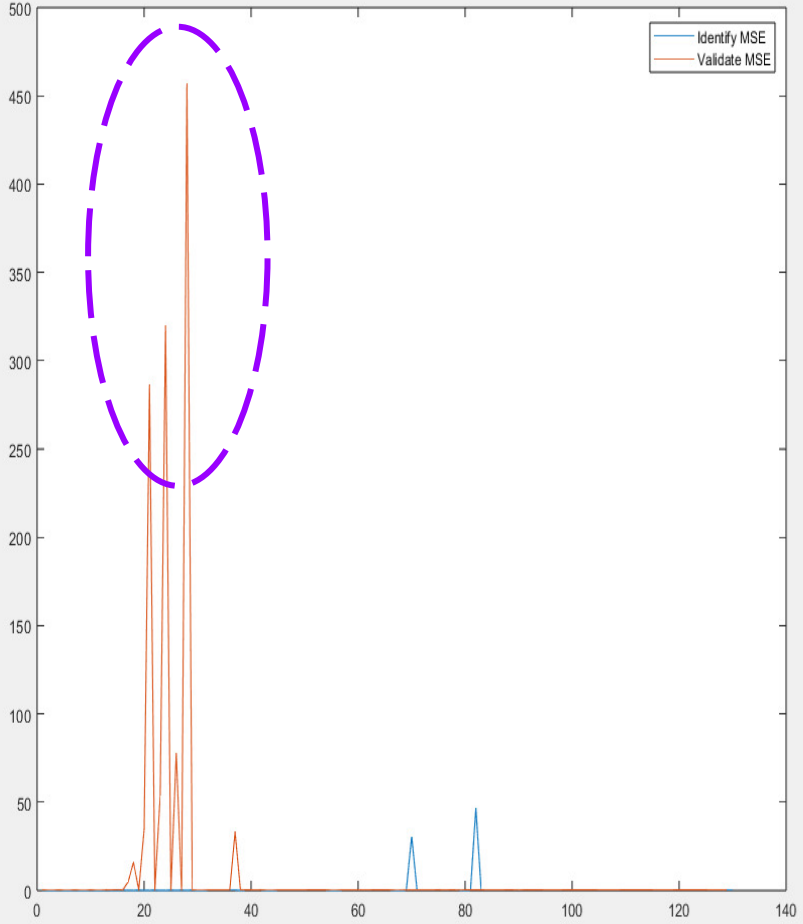
4. Cel mai optim model obținut



Eroarea medie pătratică (date originale)

- avem **EMP** pentru predicție: **0.00187** pentru **identificare**, **0.006178** pentru **validare,**
- avem **EMP** pentru simulare: **0.000871** pentru **identificare, 0.51926** pentru **validare,**
- **gradele optime** sunt : n_a **= 1**, n_b **= 1 și m = 3**





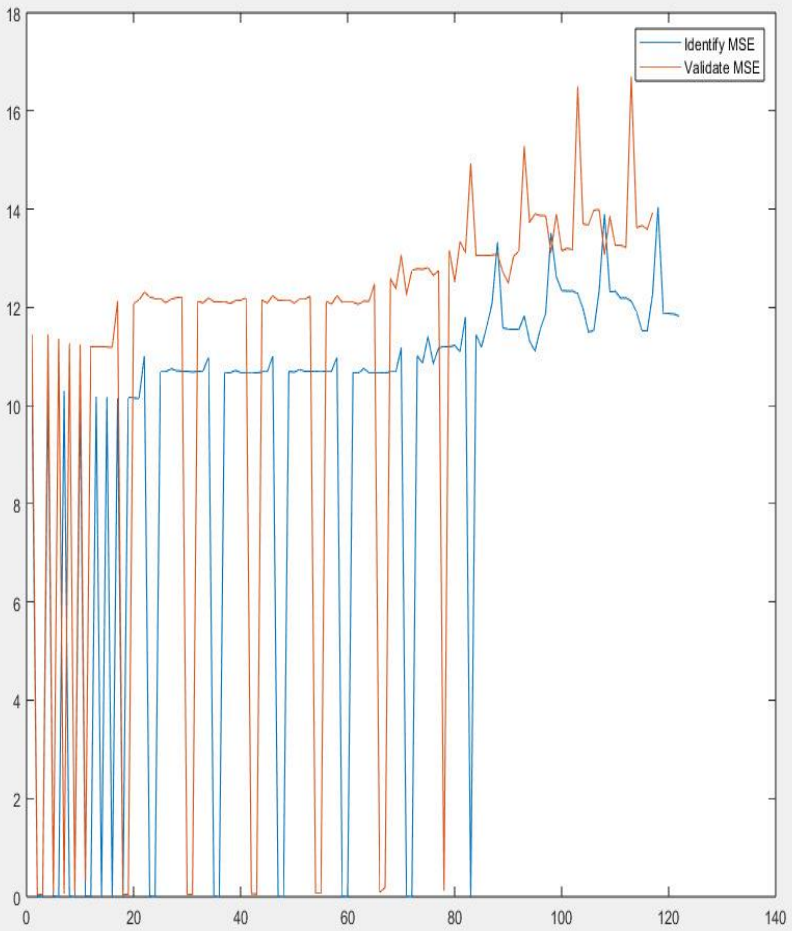
Observații

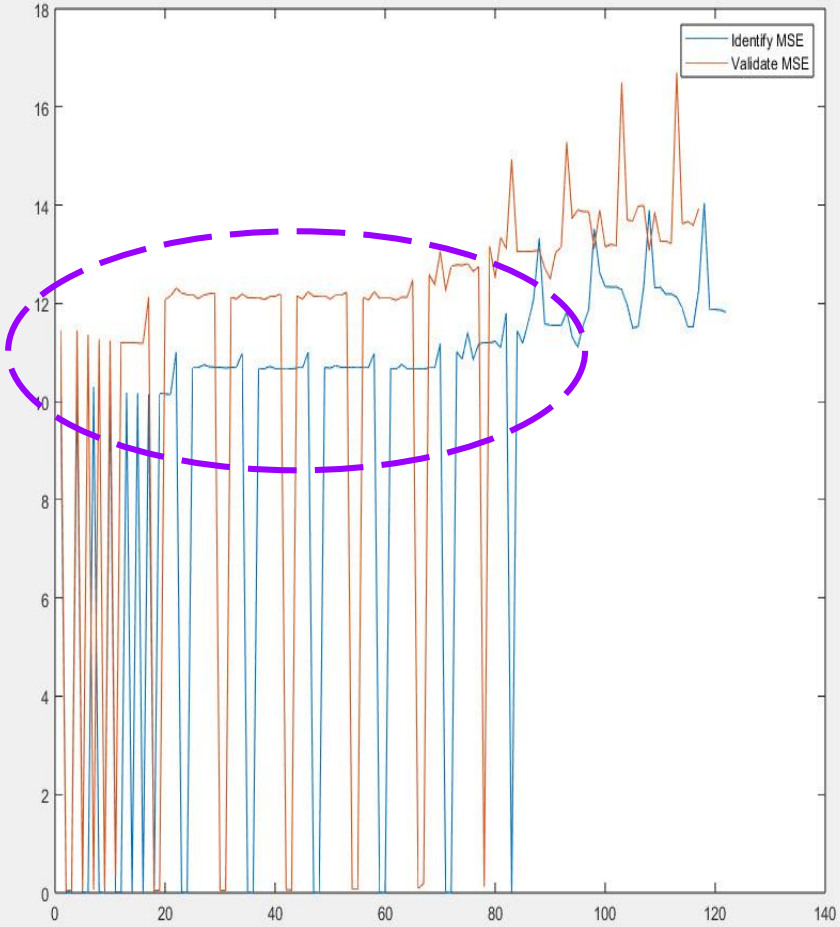
- graficul EMP-ului aferent datelor de validare prezintă o formă mai "**ciudată**" deoarece, datele nefiind normalizate, valorile prezintă **variații** destul de mari.

- din cauza acestor variații (**a valorilor regresorilor**), **sistemul** poate deveni **mai sensibil** la **perturbații.**

EMP (date normalizate prin împărțire la numărul de eșantioane)

- avem EMP pentru predicție: **0.00017** pentru **identificare**, **0.004476** pentru **validare**.
- avem EMP pentru simulare: **0.000040** pentru **identificare**, **0.035536** pentru **validare.**
- gradele optime sunt: **n_a = 1, n_b = 1 și m = 3.**





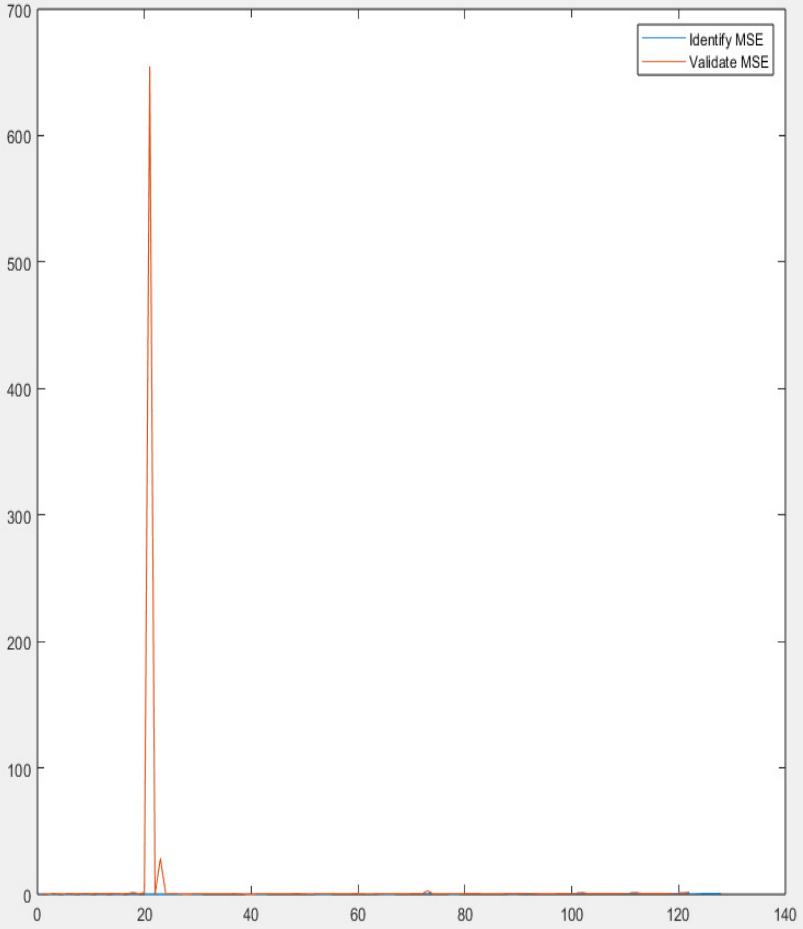
Observații

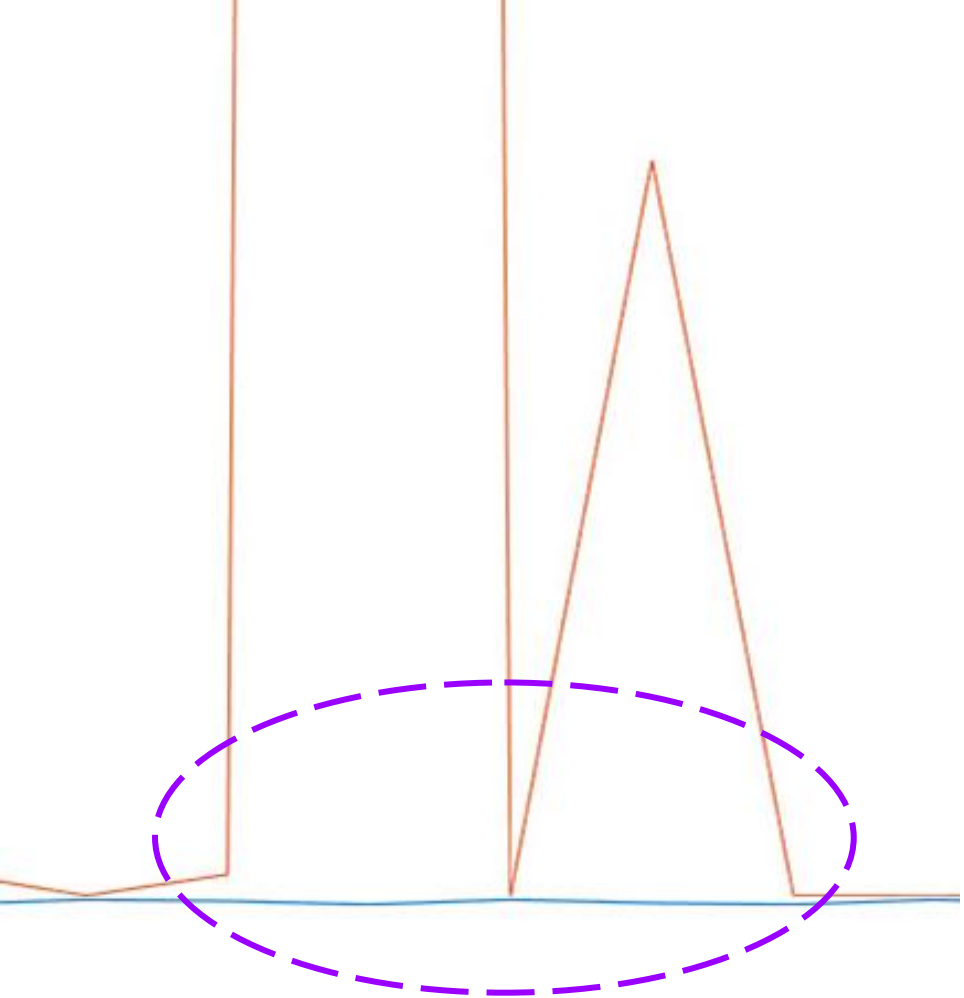
- în cazul datelor normalizate prin împărțire la **numărul de eșantioane**, variațiile sunt din nou prezente, dar mai **puțin semnificative**

- de aceea, sistemul este **mai rezistent** la **perturbații**

EMP (date normalizate prin trecerea valorilor în intervalul [-1, 1])

- avem EMP pentru predicție: **0.000352** pentru identificare, **0.003979** pentru validare.
- avem EMP pentru simulare : **0.000315** pentru identificare, **0.038303** pentru validare
- gradele optime sunt: **n_a = 3, n_b = 1 și m = 3.**





Observații

- valorile fiind **normalizate**, se reduce fenomenul de **instabilitate numerică** și **scade sensibilitatea** sistemului.

- normalizarea datelor într-un interval ales reduce **instabilitatea termenilor** și duce la **stabilizarea identificării**.

5. Concluzii

Scopul proiectului a constat în **identificarea unui sistem** utilizând un model **ARX neliniar polinomial**, utilizând diferite metode de procesare a datelor, ceea ce a dus la **rezultate diferite** referitoare la **performanța sistemului**. Metoda ARX neliniar oferă **rezultate promițătoare în predicție (dar false)**, însă **simularea** prezintă un **grad mai mare de instabilitate** în timp.

6. Anexă: cod sursă - normalizarea datelor

```
clear;
load iddata-07.mat
t = id_array(:, 1);
idInput = id_array(:, 2); valInput = val_array(:, 2);
idOutput = id_array(:, 3); valOutput = val_array(:, 3);

inputMin = min(idInput); inputMax = max(idInput);
idInput  = 2 * (idInput  - inputMin) / (inputMax - inputMin) - 1;
valInput = 2 * (valInput - inputMin) / (inputMax - inputMin) - 1;

%for i = 1 : 801
%    idInput(i) = idInput(i) / 801;
%    valInput(i) = valInput(i) / 801;
%end
```

6. Anexă: cod sursă - generarea puterilor regresorilor

```
function powers = generatePowers(currentPowers, position, remainingPositions, powers)
    if position == length(currentPowers)
        currentPowers(position) = remainingPositions;
        powers = [powers; currentPowers];
    else
        for i = remainingPositions: -1 : 0
            currentPowers(position) = i;
            powers = generatePowers(currentPowers, position + 1, remainingPositions - i
        end
    end
end

naMax = 10; nbMax = 10; nk = 1; mMax = 3;
minMSE = 1000000;
MSE = zeros(naMax * nbMax * mMax, 1);
index = 1;
for na = 1 : naMax
    for nb = 1 : nbMax
        for m = 1 : mMax
            regNumber = nchoosek(m + na + nb, na + nb);
            regPowers = []; currentPowers = zeros(1, na + nb + 1);
            regPowers = generatePowers(currentPowers, 1, m, regPowers);
```

6. Anexă: cod sursă - generarea matricei Φ și identificarea parametrilor θ

```
N = length(idInput);
phiId = zeros(N, regNumber);
for i = 1 : N
    for j = 1 : regNumber
        prod = 1^regPowers(j, 1);
        for k = 1 : na
            if(i - k > 0)
                prod = prod * idOutput(i - k)^regPowers(j, k + 1);
            end
        end
        for k = 1 : nb
            if(i - nk - k + 1 > 0)
                prod = prod * idInput(i - nk - k + 1)^regPowers(j, na + 1
            end
        end
        phiId(i, j) = prod;
    end
end
teta = phiId \ idOutput;
```

6. Anexă: cod sursă - generarea simulării și calcularea EMP

```
ySimId = zeros(N, 1); phiId = zeros(1, regNumber);
for i = 1 : N
    for j = 1 : regNumber
        prod = 1^regPowers(j, 1);
        for k = 1 : na
            if i - k > 0
                prod = prod * ySimId(i - k)^regPowers(j, k + 1);
            end
        end
        for k = 1 : nb
            if i - nk - k + 1 > 0
                prod = prod * idInput(i - nk - k + 1)^regPowers(j, na + 1 + k);
            end
        end
        phiId(j) = prod;
    end
    ySimId(i) = phiId * teta;
end

ySimVal = zeros(N, 1); phiVal = zeros(1, regNumber);
for i = 1 : N
    for j = 1 : regNumber
        prod = 1^regPowers(j, 1);
        for k = 1 : na
            if i - k > 0
                prod = prod * ySimVal(i - k)^regPowers(j, k + 1);
            end
        end
        for k = 1 : nb
            if i - nk - k + 1 > 0
                prod = prod * valInput(i - nk - k + 1)^regPowers(j, na + 1 + k);
            end
        end
        phiVal(j) = prod;
    end
    ySimVal(i) = phiVal * teta;
end
```

```
mseSimId = mean((idOutput - ySimId).^2);
mseSimVal = mean((valOutput - ySimVal).^2);
MSEid(index) = mseSimId;
MSEval(index) = mseSimVal;
fprintf("%d - MSE id: %f | MSE val: %f | na:
index = index + 1;
if mseSimVal < minMSE
    bestYSim = ySimVal;
    minMSE = mseSimVal;
    bestNa = na;
    bestNb = nb;
    bestM = m;
    bestTeta = teta;
    bestRegPowers = regPowers;
end
```

6. Anexă: cod sursă - generarea predicției pentru gradele optime

```
yPredId = zeros(N, 1); phiId = zeros(1, regNumber);
for i = 1 : N
    for j = 1 : regNumber
        prod = 1^regPowers(j, 1);
        for k = 1 : na
            if i - k > 0
                prod = prod * idOutput(i - k)^regPowers(j, k + 1);
            end
        end
        for k = 1 : nb
            if i - nk - k + 1 > 0
                prod = prod * idInput(i - nk - k + 1)^regPowers(j, na + 1 + k);
            end
        end
        phiId(j) = prod;
    end
    yPredId(i) = phiId * teta;
end
yPredVal = zeros(N, 1); phiVal = zeros(1, regNumber);
for i = 1 : N
    for j = 1 : regNumber
        prod = 1^regPowers(j, 1);
        for k = 1 : na
            if i - k > 0
                prod = prod * valOutput(i - k)^regPowers(j, k + 1);
            end
        end
        for k = 1 : nb
            if i - nk - k + 1 > 0
                prod = prod * valInput(i - nk - k + 1)^regPowers(j, na + 1 + k);
            end
        end
        phiVal(j) = prod;
    end
    yPredVal(i) = phiVal * teta;
end
```

6. Anexă: cod sursă - afișarea rezultatelor

```
yPred = iddata(yPredVal, valInput, id.Ts);
ySim = iddata(bestYSim, valInput, id.Ts);
figure;
compare(val, yPred, ySim);

fprintf('Best na = %d, nb = %d, m = %d\n', na, nb, m);
fprintf('MSE Prediction Identify: %f | MSE Prediction Validate: %f\n', predMSEId, predMSEVal);
fprintf('MSE Simulation Identify: %f | MSE Simulation Validate: %f\n', min(MSEid), min(MSEval));

filteredMSEid = zeros(length(MSEid), 1);
index = 1; nanValuesId = 0;
for i = 1 : length(MSEid)
    if isnan(MSEid(i))
        nanValuesId = nanValuesId + 1;
        continue;
    end
    filteredMSEid(index) = MSEid(i);
    index = index + 1;
end

filteredMSEval = zeros(length(MSEval), 1);
index = 1; nanValuesVal = 0;
for i = 1 : length(MSEval)
    if isnan(MSEval(i))
        nanValuesVal = nanValuesVal + 1;
        continue;
    end
    filteredMSEval(index) = MSEval(i);
    index = index + 1;
end
figure;
plot(filteredMSEid(1 : length(MSEid) - nanValuesId));
hold on;
plot(filteredMSEval(1 : length(MSEval) - nanValuesVal));
legend("Identify MSE", "Validate MSE");
```